QUANTINUUM

ReTrace:

Resource Estimation via the Logical Execution **Trace**

The 8th International Conference on Quantum Resource Estimation (QRE26)

Tyler LeBlond

© 2026 Quantinuum. All Rights Reserved.

August 24th, 2026

Resource Estimation for Fault-Tolerant Quantum Computers (FTQC)

Why do we need it?

- Benchmarking: evaluate the theoretical performance of quantum algorithms on our future devices
- Code selection: discern what FTQC protocols lead to better performance for those algorithms
- Compilation and scheduling: enable efficient compilation and scheduling for applications of FTQC

Public
© 2026 Quantinuum. All Rights Reserved.

Resource Estimation for Fault-Tolerant Quantum Computers (FTQC)

Why do we need it?

- Benchmarking: evaluate the theoretical performance of quantum algorithms on our future devices
- Code selection: discern what FTQC protocols lead to better performance for those algorithms
- Compilation and scheduling: enable efficient compilation and scheduling for applications of FTQC

Existing open-source RE tools are (i) *pessimistic for ions* and (ii) *back-of-the-envelope*

© 2026 Quantinuum. All Rights Reserved.

Resource Estimation for Fault-Tolerant Quantum Computers (FTQC)

Why do we need it?

- Benchmarking: evaluate the theoretical performance of quantum algorithms on our future devices
- Code selection: discern what FTQC protocols lead to better performance for those algorithms
- Compilation and scheduling: enable efficient compilation and scheduling for applications of FTQC

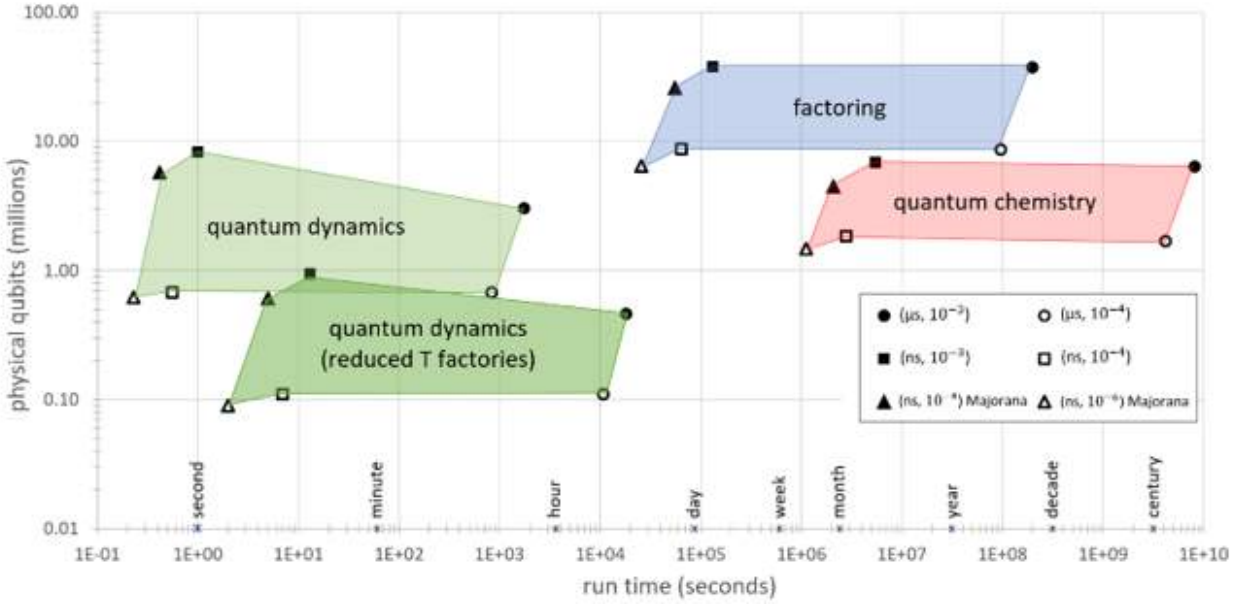
Existing open-source RE tools are (i) *pessimistic for ions* and (ii) *back-of-the-envelope*

Microsoft's Quantum Resource Estimator tool:
[Beverland et. al., arXiv:2211.07629]

- Imposes *nearest-neighbor connectivity* constraint, *even for ions*
- *Surface code lattice surgery* architecture
- Uses (a refined version of) *Pauli-based computation*

Resource estimates back-of-the-envelope:

- **Assumes** whole computer is always allocated and always being error corrected in every cycle
- **Assumes** constant rate of magic state consumption throughout circuit execution
- **Neglects** overheads of parallelism (logical ancilla, magic state transport, factory failure)

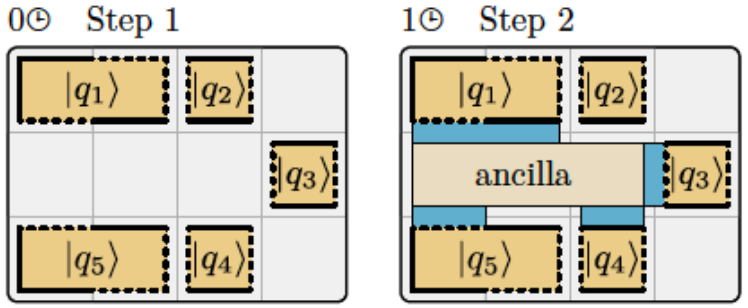


© 2026 Quantinuum. All Rights Reserved.

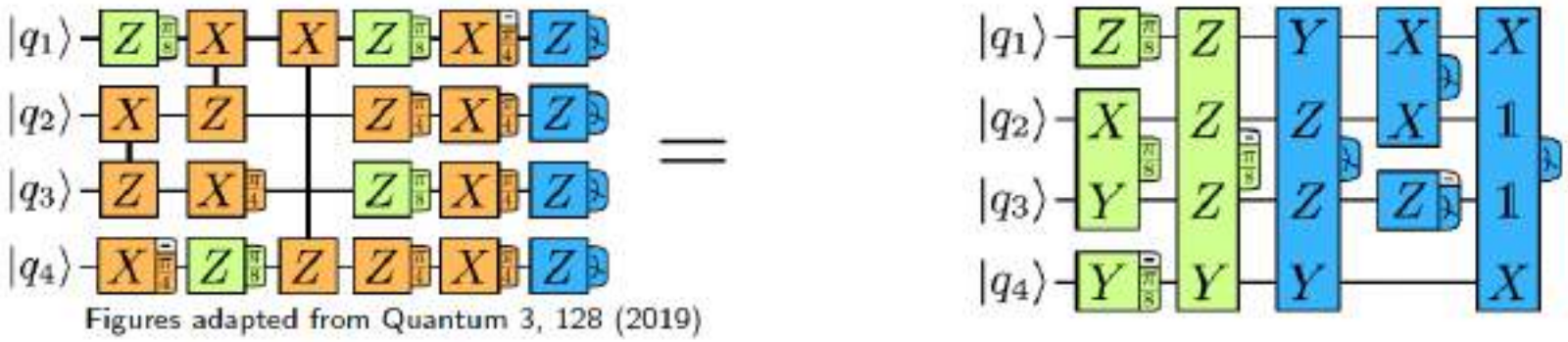
Surface Code Lattice Surgery

Necessary for nearest-neighbor connected architectures

- The surface code has a native Pauli product measurement instruction set based on lattice surgery
- Each lattice surgery operation requires O(d) syndrome extraction cycles due to measurement errors



- Quantum circuits can be compiled to this instruction set at the expense of serialization



- Because of this serialization, the *logical depth-1 circuit time** is $O(n_L d)$

© 2026 Quantinuum. All Rights Reserved.

Surface Code Lattice Surgery

Necessary for nearest-neighbor connected architectures

- Alternatively, Clifford+T circuits can be directly compiled using Bell-based communication to implement long-range CNOT gates

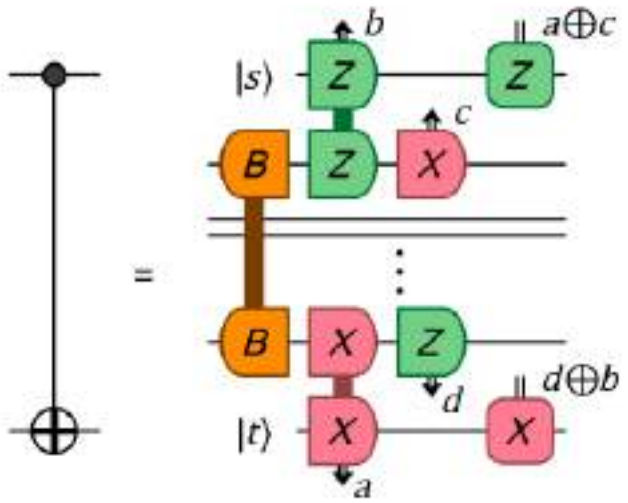
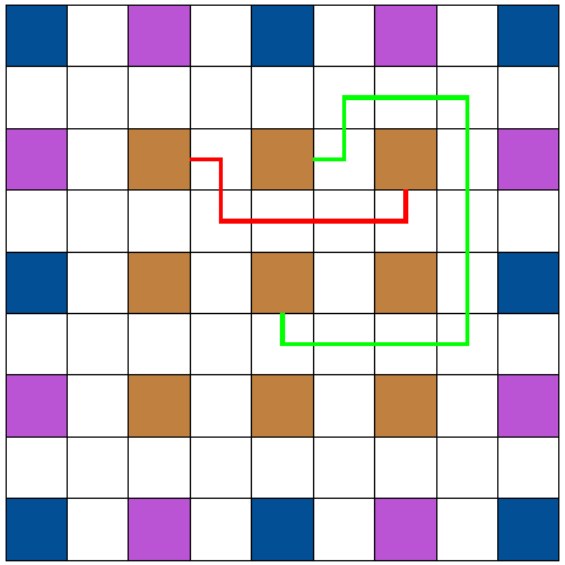
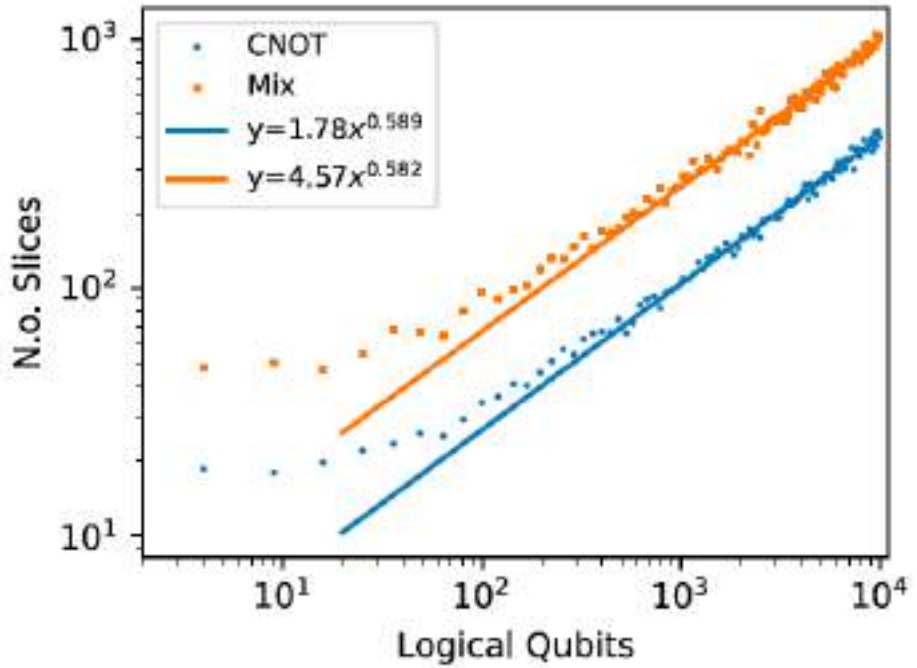


Fig. adapted from PRX Quantum 3 020342 (2022)





LeBlond *et. al.* 2024

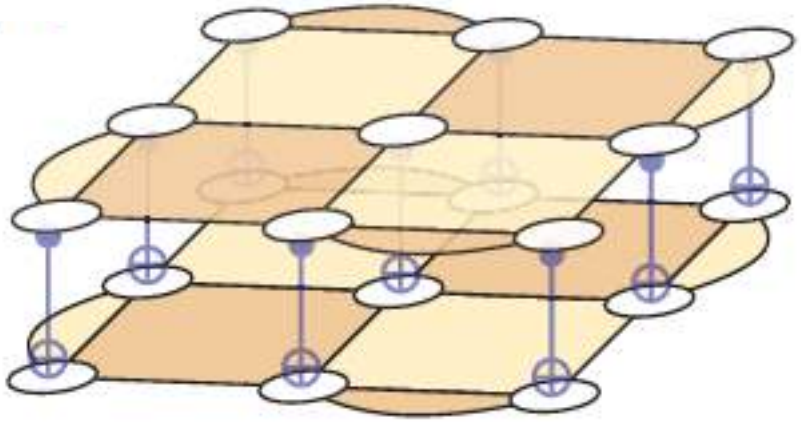
- While not sequential, this scheme incurs an O($\sqrt{n_L}$) overhead in the number of lattice surgery slices to implement a layer of random CNOT gates, e.g. the *logical depth-1 circuit time is O($\sqrt{n_L}d$)*

© 2026 Quantinuum. All Rights Reserved.

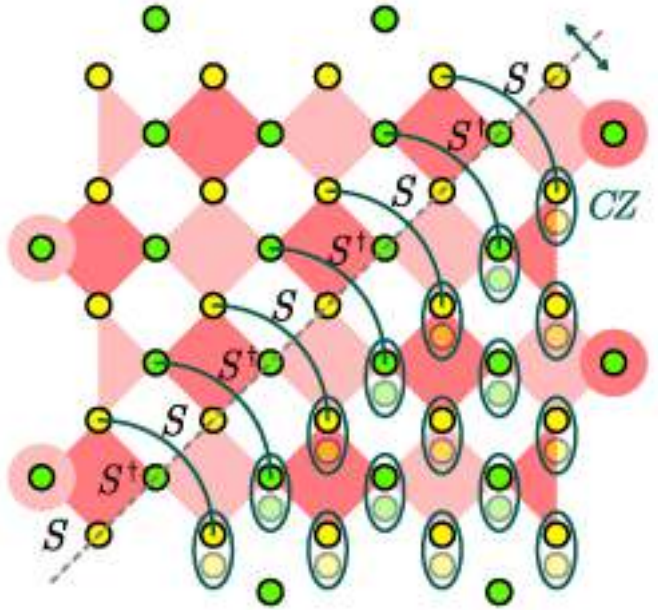
Surface Code with Transversal Gates

An option only for all-to-all connected architectures

- Transversal CNOT, H, and S enabled by all-to-all connectivity



Adapted from PRX Quantum **6**, 020326



Adapted from PRL **136**, 130601

- Correlated decoding enables the avoidance of O(d) rounds of syndrome extraction per logical gate
  - Cain et. al., PRL **133**, 240602, Cain et. al., arXiv:2505.13587
- With true all-to-all connectivity, the *logical depth-1 circuit time* becomes O(1)!

© 2026 Quantinuum. All Rights Reserved.

Resource Estimation for *All-to-All Connected* FTQC

Public

© 2026 Quantinuum. All Rights Reserved.

Resource Estimation for *All-to-All Connected* FTQC

Our approach for all-to-all connected architectures

Fix the hardware model: total # physical qubits, physical error rates, depth-1 circuit time, logical error rate

Inputs: ‘resources’ for individual FT gadgets (# physical qubits, depth, logical error rate, success rate, etc.)

Inputs: ‘computational’ quantum program (expressed in, e.g., Clifford+T)



Outputs: time-to-solution, total logical error rate, post-selection rate, detailed instruction schedule, resource allocation profiles, distribution of scheduled instructions over time, circuit completion over time

© 2026 Quantinuum. All Rights Reserved.

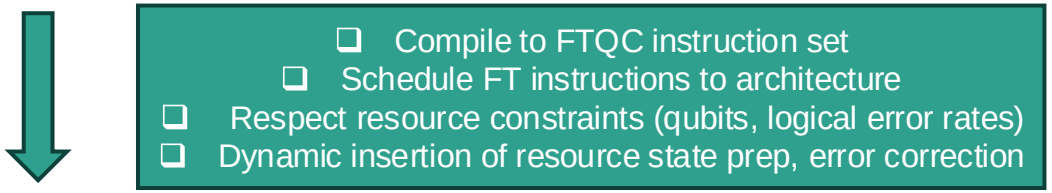
Resource Estimation for *All-to-All Connected* FTQC

Our approach for all-to-all connected architectures

Fix the hardware model: total # physical qubits, physical error rates, depth-1 circuit time, logical error rate

Inputs: ‘resources’ for individual FT gadgets (# physical qubits, depth, logical error rate, success rate, etc.)

Inputs: ‘computational’ quantum program (expressed in, e.g., Clifford+T)



Outputs: time-to-solution, total logical error rate, post-selection rate, detailed instruction schedule, resource allocation profiles, distribution of scheduled instructions over time, circuit completion over time

ReTrace goes significantly beyond ‘traditional’ approaches:

- Real logical compiler/code agnostic scheduler pipeline with many ‘real-time’/dynamic aspects:

© 2026 Quantinuum. All Rights Reserved.

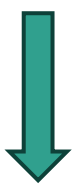
Resource Estimation for *All-to-All Connected* FTQC

Our approach for all-to-all connected architectures

Fix the hardware model: total # physical qubits, physical error rates, depth-1 circuit time, logical error rate

Inputs: ‘resources’ for individual FT gadgets (# physical qubits, depth, logical error rate, success rate, etc.)

Inputs: ‘computational’ quantum program (expressed in, e.g., Clifford+T)



- ❑ Compile to FTQC instruction set
- ❑ Schedule FT instructions to architecture
- ❑ Respect resource constraints (qubits, logical error rates)
- ❑ Dynamic insertion of resource state prep, error correction

Outputs: time-to-solution, total logical error rate, post-selection rate, detailed instruction schedule, resource allocation profiles, distribution of scheduled instructions over time, circuit completion over time

ReTrace goes significantly beyond ‘traditional’ approaches:

- Real logical compiler/code agnostic scheduler pipeline with many ‘real-time’/dynamic aspects:
  - Explicitly handles non-determinism (failing state preparation, conditional Clifford corrections, etc.)

© 2026 Quantinuum. All Rights Reserved.

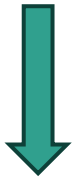
Resource Estimation for *All-to-All Connected* FTQC

Our approach for all-to-all connected architectures

Fix the hardware model: total # physical qubits, physical error rates, depth-1 circuit time, logical error rate

Inputs: ‘resources’ for individual FT gadgets (# physical qubits, depth, logical error rate, success rate, etc.)

Inputs: ‘computational’ quantum program (expressed in, e.g., Clifford+T)



- ❑ Compile to FTQC instruction set
- ❑ Schedule FT instructions to architecture
- ❑ Respect resource constraints (qubits, logical error rates)
- ❑ Dynamic insertion of resource state prep, error correction

Outputs: time-to-solution, total logical error rate, post-selection rate, detailed instruction schedule, resource allocation profiles, distribution of scheduled instructions over time, circuit completion over time

ReTrace goes significantly beyond ‘traditional’ approaches:

- Real logical compiler/code agnostic scheduler pipeline with many ‘real-time’/dynamic aspects:
  - Explicitly handles non-determinism (failing state preparation, conditional Clifford corrections, etc.)
  - Instructions scheduled according to dynamic set of available ‘heads’ and respect ‘real-time’ constraints

© 2026 Quantinuum. All Rights Reserved.

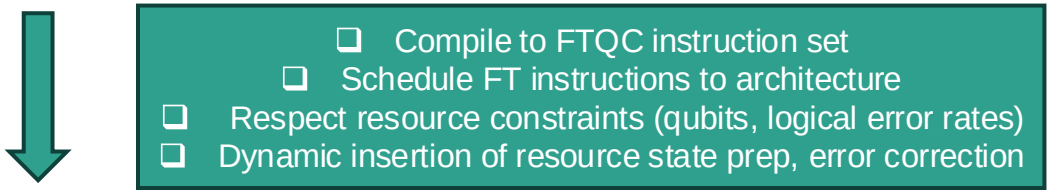
Resource Estimation for *All-to-All Connected* FTQC

Our approach for all-to-all connected architectures

Fix the hardware model: total # physical qubits, physical error rates, depth-1 circuit time, logical error rate

Inputs: ‘resources’ for individual FT gadgets (# physical qubits, depth, logical error rate, success rate, etc.)

Inputs: ‘computational’ quantum program (expressed in, e.g., Clifford+T)



Outputs: time-to-solution, total logical error rate, post-selection rate, detailed instruction schedule, resource allocation profiles, distribution of scheduled instructions over time, circuit completion over time

ReTrace goes significantly beyond ‘traditional’ approaches:

- Real logical compiler/code agnostic scheduler pipeline with many ‘real-time’/dynamic aspects:
  - Explicitly handles non-determinism (failing state preparation, conditional Clifford corrections, etc.)
  - Instructions scheduled according to dynamic set of available ‘heads’ and respect ‘real-time’ constraints
  - Resource state production based on instantaneous demand, expiry when ‘stale’ or perform error correction
    - Explicit tracking of of active code block lifetimes (data, resource states)

© 2026 Quantinuum. All Rights Reserved.

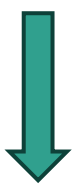
Resource Estimation for *All-to-All Connected* FTQC

Our approach for all-to-all connected architectures

Fix the hardware model: total # physical qubits, physical error rates, depth-1 circuit time, logical error rate

Inputs: ‘resources’ for individual FT gadgets (# physical qubits, depth, logical error rate, success rate, etc.)

Inputs: ‘computational’ quantum program (expressed in, e.g., Clifford+T)



- ❑ Compile to FTQC instruction set
- ❑ Schedule FT instructions to architecture
- ❑ Respect resource constraints (qubits, logical error rates)
- ❑ Dynamic insertion of resource state prep, error correction

Outputs: time-to-solution, total logical error rate, post-selection rate, detailed instruction schedule, resource allocation profiles, distribution of scheduled instructions over time, circuit completion over time

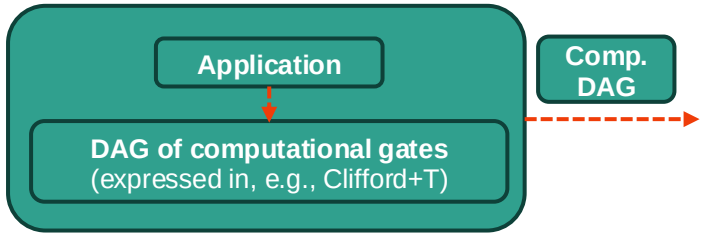
ReTrace goes significantly beyond ‘traditional’ approaches:

- Real logical compiler/code agnostic scheduler pipeline with many ‘real-time’/dynamic aspects:
  - Explicitly handles non-determinism (failing state preparation, conditional Clifford corrections, etc.)
  - Instructions scheduled according to dynamic set of available ‘heads’ and respect ‘real-time’ constraints
  - Resource state production based on instantaneous demand, expiry when ‘stale’ or perform error correction
    - Explicit tracking of of active code block lifetimes (data, resource states)
  - Error correction insertion based on ‘real-time’ logical error rate with adaptive schedule

Public

© 2026 Quantinuum. All Rights Reserved.

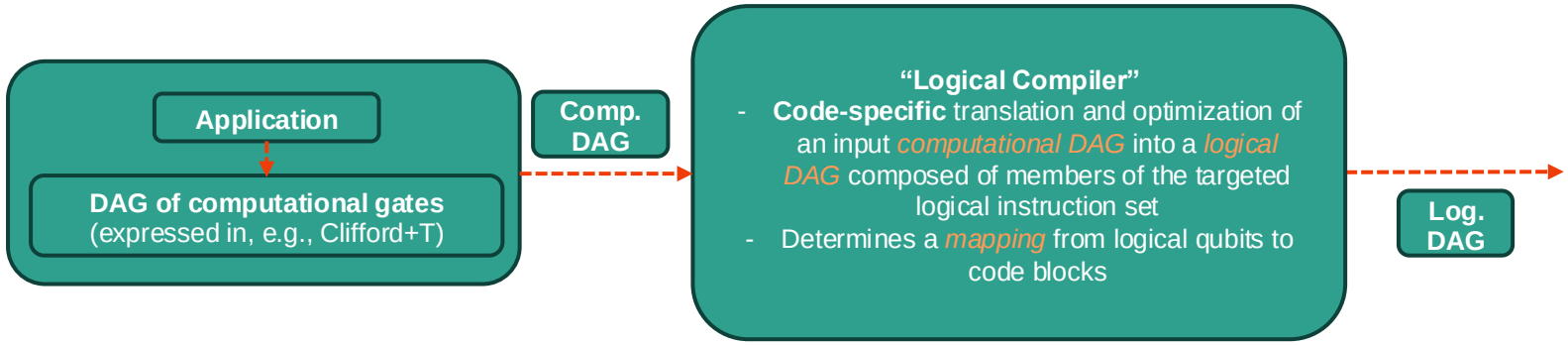
ReTrace: high-level design of framework



* DAG = directed acyclic graph (a quantum circuit representation)

© 2026 Quantinuum. All Rights Reserved.

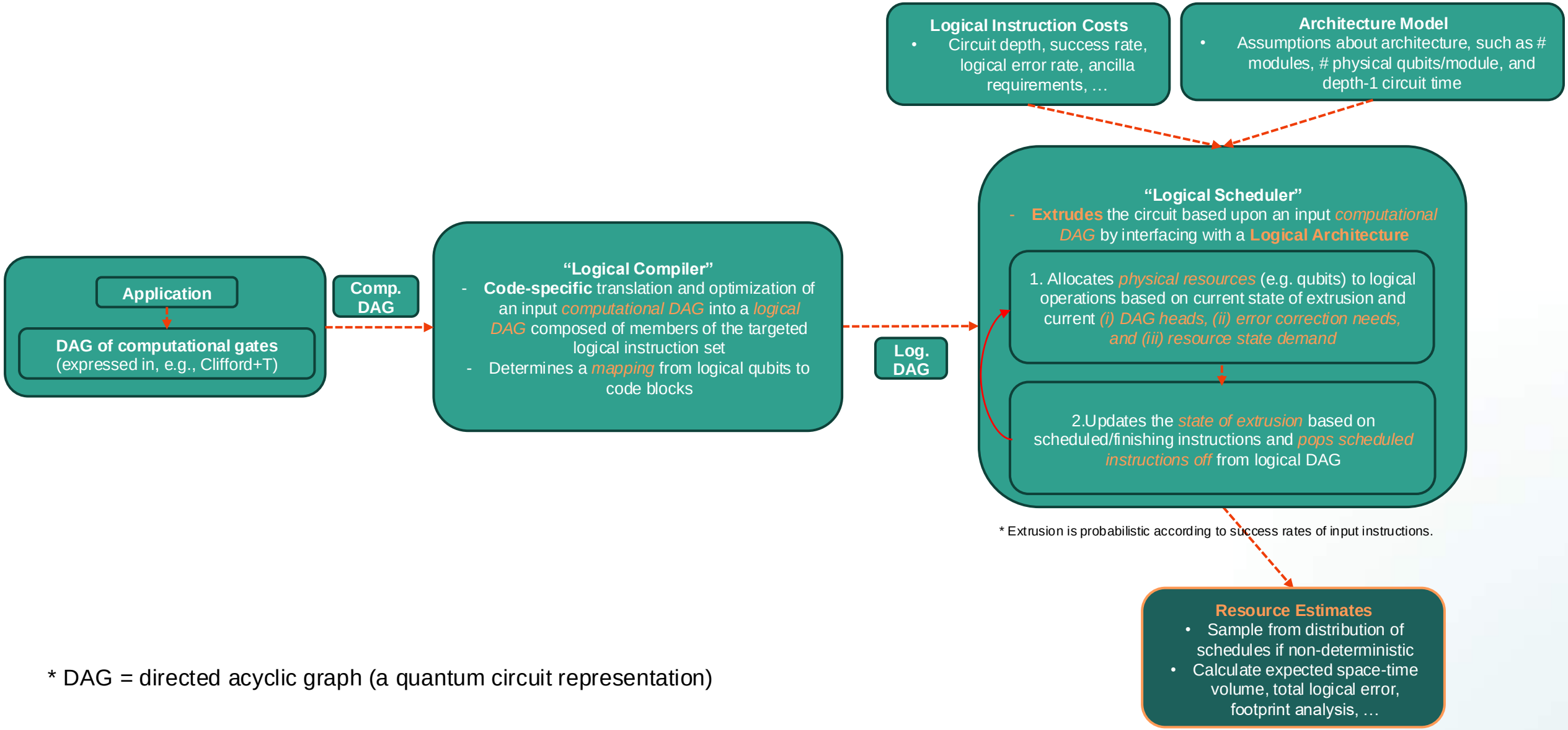
ReTrace: high-level design of framework



* DAG = directed acyclic graph (a quantum circuit representation)

© 2026 Quantinuum. All Rights Reserved.

ReTrace: high-level design of framework



* DAG = directed acyclic graph (a quantum circuit representation)

© 2026 Quantinuum. All Rights Reserved.

ReTrace: Logical Compilers

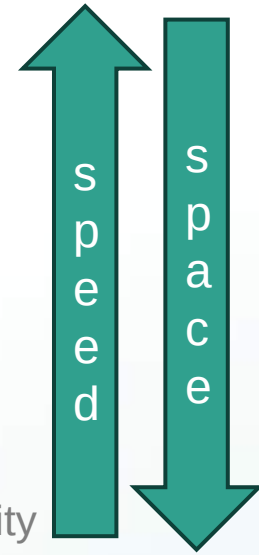
Input circuit transpilation/resynthesis

A *code-specific* tool that transforms an input circuit into a circuit composed of FT operations.

Our implementations are purely static and *do not* take resource overheads, scheduling, parallelism into account (that is the job of the **Scheduler**, a different tool in the pipeline).

We have developed independent logical compilers for a few QEC codes:

- Surface code/color code
  - Transversal CNOT, H, (fold-)transversal S, inject T, magic state cultivation
- [[20,2,6]] $C_4 - \mathrm{Helix}$ code arXiv:2609.03194
  - Whole intra-block Clifford group implementable through automorphisms and global S gates
- Hypercube-concatenated [[4,2,2]] code Goto, Sci. Adv. **10** (2024)**,** arXiv:2508.05779
  - Batches of CNOT gates using automorphisms, inject global T and S, special mappings promote transversality



Desired:

- High-rate qLDPC code(s) using parallel lattice surgery techniques
- Basic research needed to work out these schemes in greater detail!

© 2026 Quantinuum. All Rights Reserved.

ReTrace: Logical Scheduler

General algorithm

- The *scheduler* is the central, *code-agnostic* component of the RE toolchain
- General assumptions: depth only counts CNOT gates, constant depth-1 circuit time, no sense of locality (any operation can grab any free physical qubit from anywhere in the quantum computer)
- Order of priority in scheduling: (1) Error Correction, (2) Circuit Instructions (DAG), (3) Resource States

Public
© 2026 Quantinuum. All Rights Reserved.

ReTrace: Logical Scheduler

General algorithm

- The *scheduler* is the central, *code-agnostic* component of the RE toolchain
- General assumptions: depth only counts CNOT gates, constant depth-1 circuit time, no sense of locality (any operation can grab any free physical qubit from anywhere in the quantum computer)
- Order of priority in scheduling: (1) Error Correction, (2) Circuit Instructions (DAG), (3) Resource States
- (Simplified version of) greedy algorithm:

  While DAG instructions remain:

  - EC resource demand <- try scheduling EC on code blocks that need it
  - EC resource state prep scheduled
  - DAG resource demand <- try scheduling DAG heads
  - DAG resource state prep scheduled
  - Advance extrusion time step if no depth-zero instruction scheduled
  - Free completing instructions (after evaluating success), add heads, and register follow-ups (if applicable)
  - Collect “garbage” (data or resource blocks)

Public
© 2026 Quantinuum. All Rights Reserved.

ReTrace: Logical Scheduler

General algorithm

- The *scheduler* is the central, *code-agnostic* component of the RE toolchain
- General assumptions: depth only counts CNOT gates, constant depth-1 circuit time, no sense of locality (any operation can grab any free physical qubit from anywhere in the quantum computer)
- Order of priority in scheduling: (1) Error Correction, (2) Circuit Instructions (DAG), (3) Resource States
- (Simplified version of) greedy algorithm:

  While DAG instructions remain:

  - EC resource demand <- try scheduling EC on code blocks that need it
  - EC resource state prep scheduled
  - DAG resource demand <- try scheduling DAG heads
  - DAG resource state prep scheduled
  - Advance extrusion time step if no depth-zero instruction scheduled
  - Free completing instructions (after evaluating success), add heads, and register follow-ups (if applicable)
  - Collect “garbage” (data or resource blocks)

- Lots of detail involved each step; I will zoom in on a couple of important features

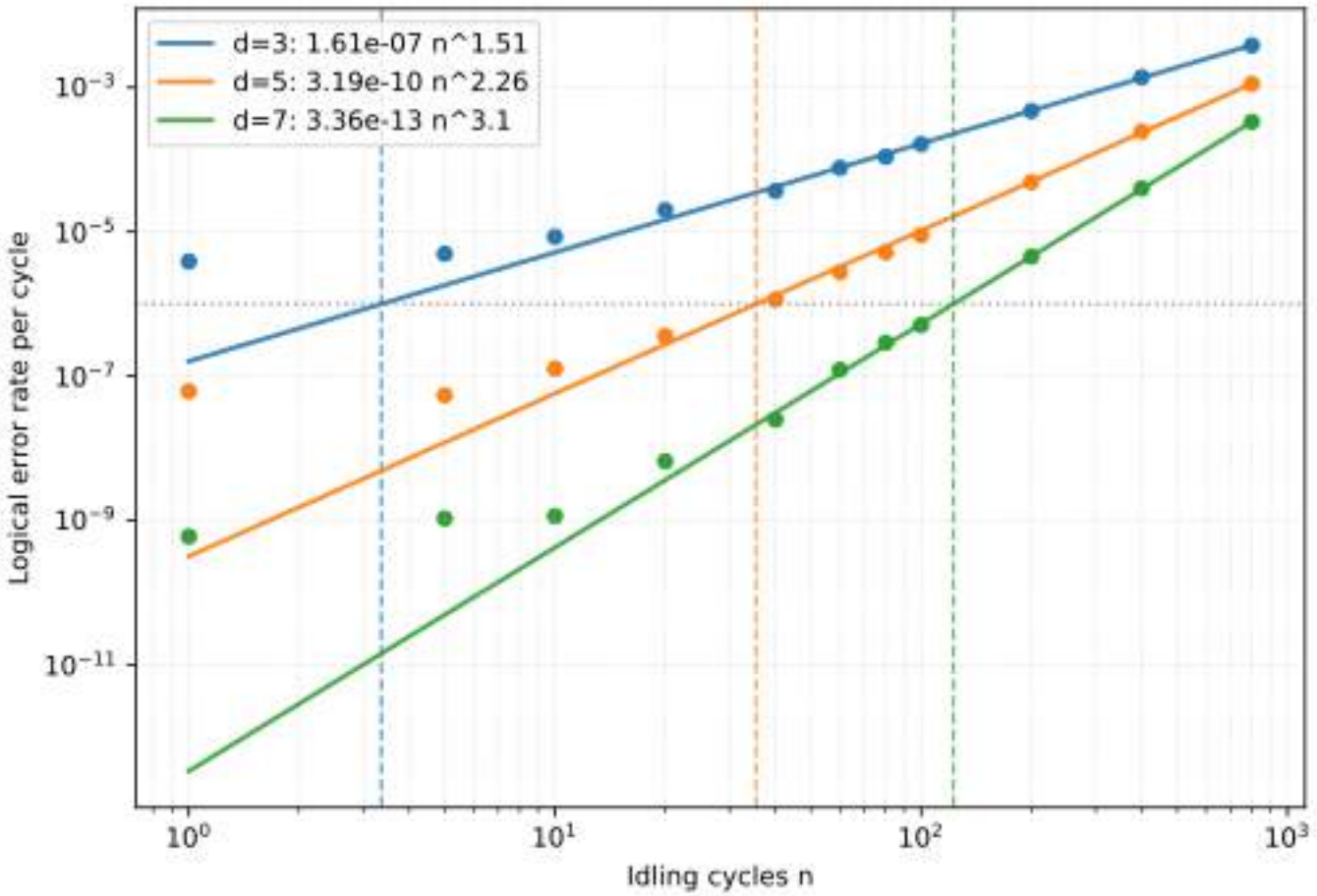
© 2026 Quantinuum. All Rights Reserved.

When to schedule error correction?

Example: Surface Code

- We don’t need to error correct every code block in parallel in every logical cycle; in fact this would do more harm than good
- Plots like these inform our estimates of the maximum tolerable number of depth-1 cycles before error correction must be applied
- Example: distance 5 surface code crosses the per-cycle LER of 1e-6 at 35 cycles
- In **ReTrace**, we guarantee* EC (for tracked code blocks) by a certain maximum idling time

  *Over-provision of resource state factories, careful reservation of physical qubits, and prioritization of EC makes the “guarantee”

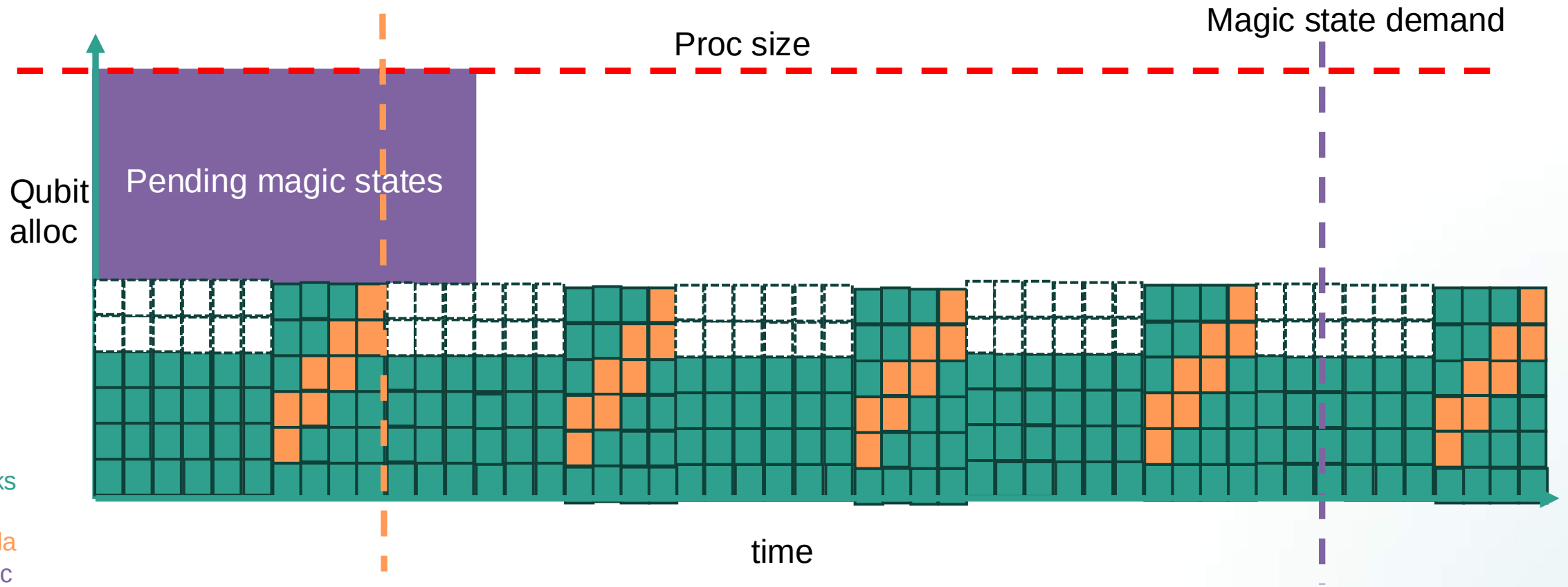


Public

© 2026 Quantinuum. All Rights Reserved.

ReTrace: Adaptive Error Correction Scheduling

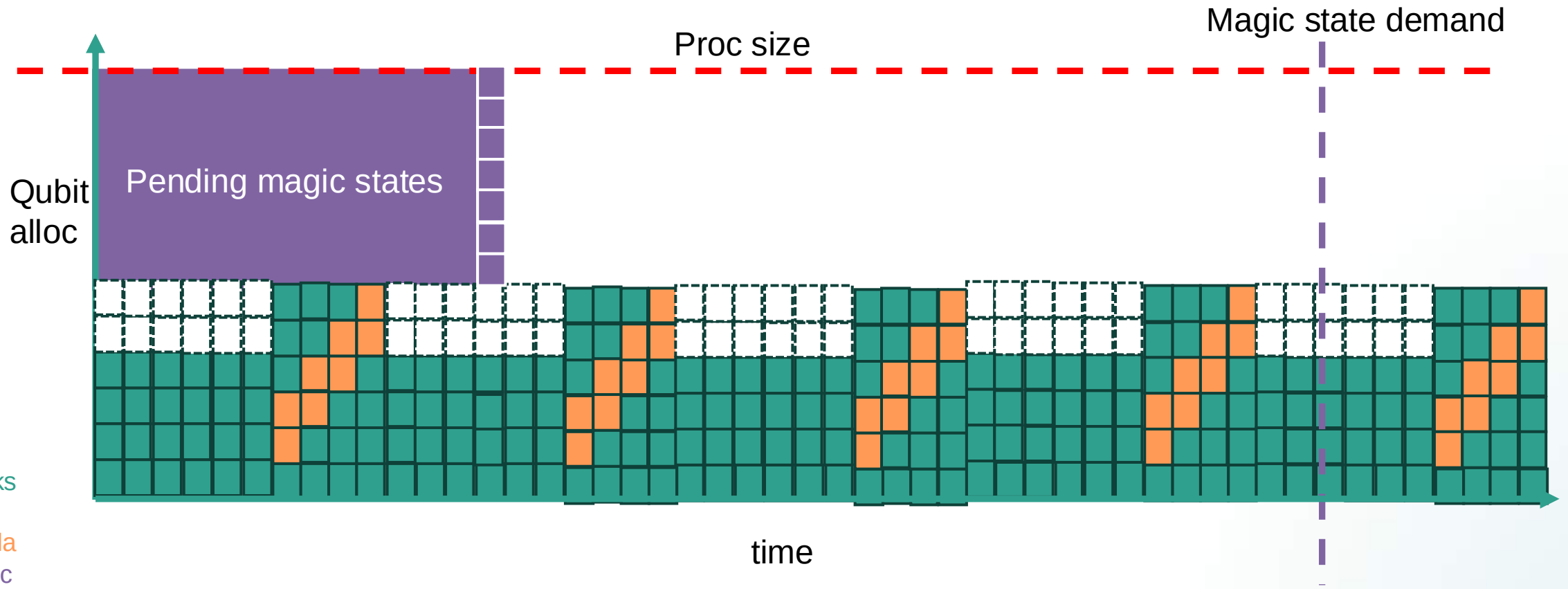
- Error correction schedule adapts with allocated code blocks, including “tracked” resource states
- Throughput can change over time (not depicted), changing spatial resource requirements
- From any given code block’s “frame”, error correction (ideally) happens as late as possible



© 2026 Quantinuum. All Rights Reserved.

ReTrace: Adaptive Error Correction Scheduling

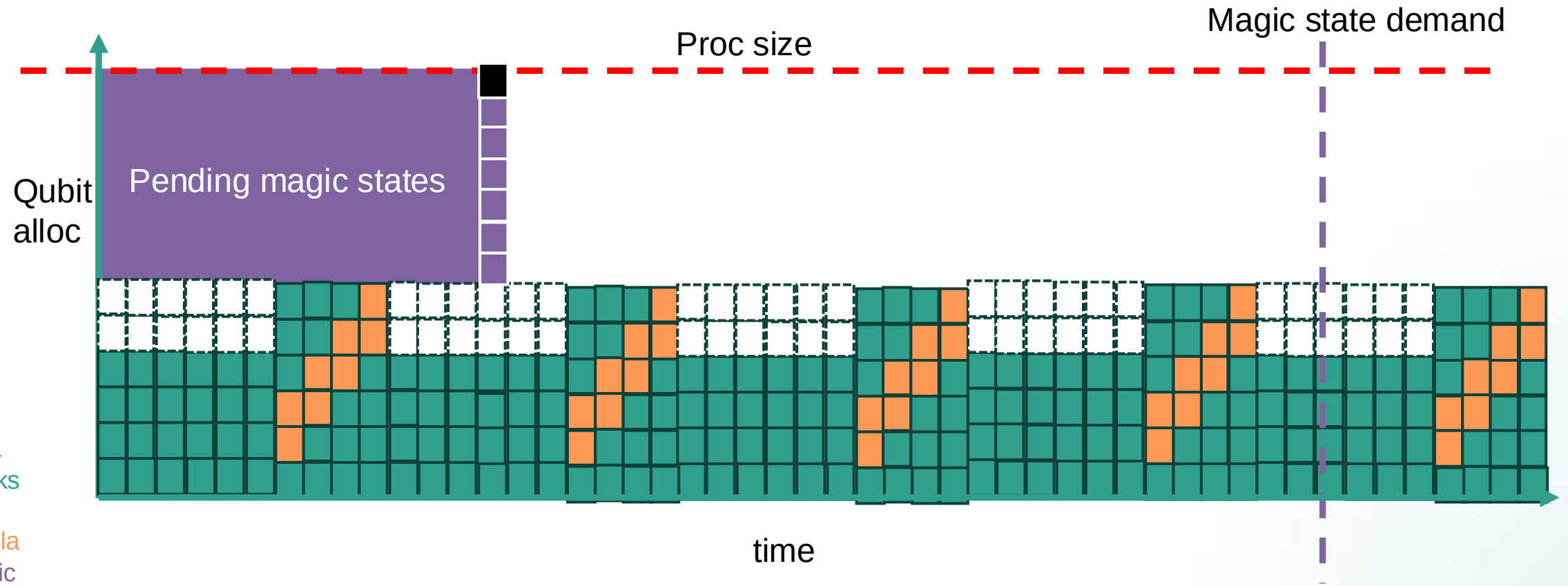
- Error correction schedule adapts with allocated code blocks, including “tracked” resource states
- Throughput can change over time (not depicted), changing spatial resource requirements
- From any given code block’s “frame”, error correction (ideally) happens as late as possible



© 2026 Quantinuum. All Rights Reserved.

ReTrace: Adaptive Error Correction Scheduling

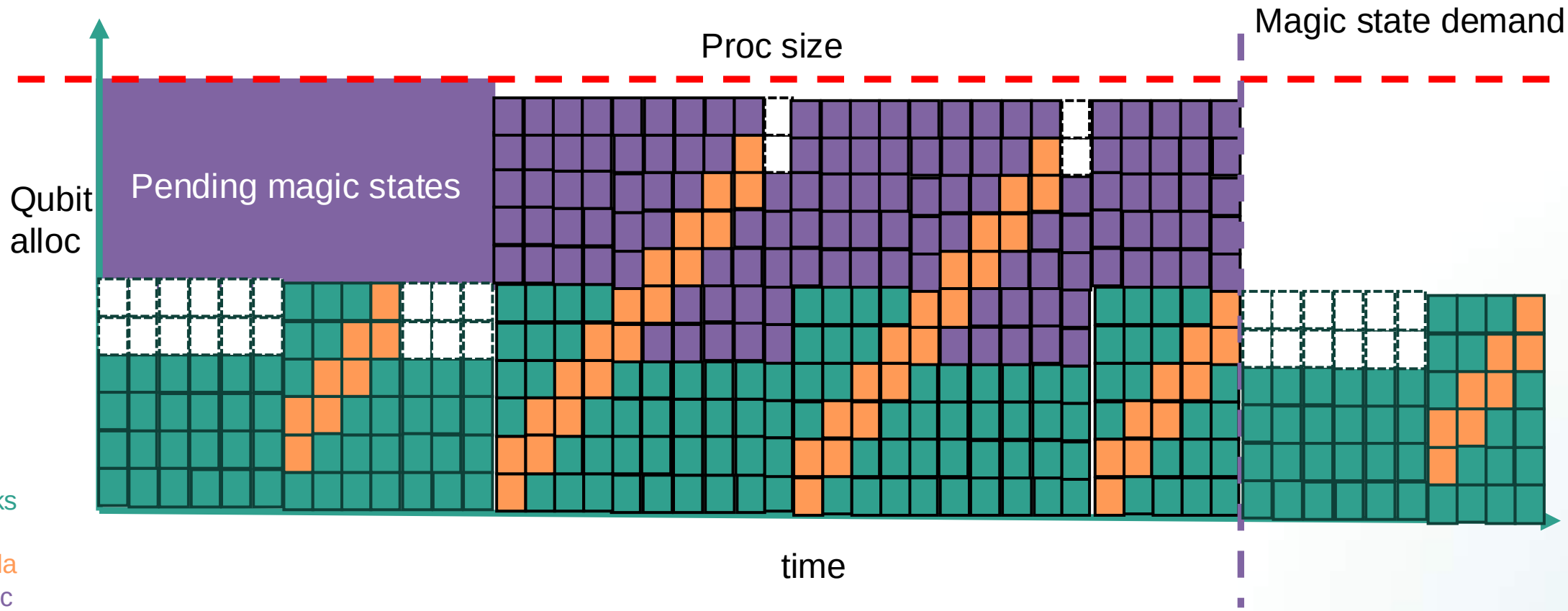
- Error correction schedule adapts with allocated code blocks, including “tracked” resource states
- Throughput can change over time (not depicted), changing spatial resource requirements
- From any given code block’s “frame”, error correction (ideally) happens as late as possible



© 2026 Quantinuum. All Rights Reserved.

ReTrace: Adaptive Error Correction Scheduling

- Error correction schedule adapts with allocated code blocks, including “tracked” resource states
- Throughput can change over time (not depicted), changing spatial resource requirements
- From any given code block’s “frame”, error correction (ideally) happens as late as possible

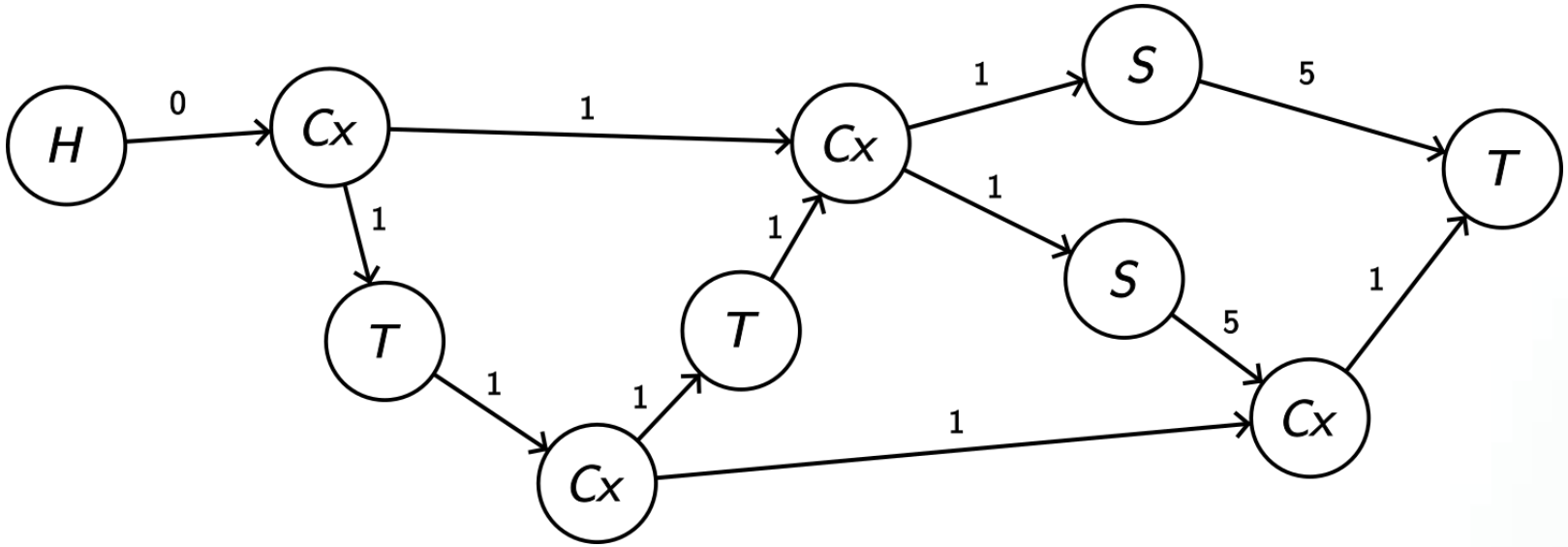


© 2026 Quantinuum. All Rights Reserved.

ReTrace: Scheduling Resource States

Demand-based resource state production scheduling

- Current default mode: *reactive* demand scheduling (no lookahead)
- The maximum number of preparations scheduled in each cycle guarantees demand met with > 99.9% prob.
- If can’t allocate that number, re-try whenever qubits become available or demand shifts
- Moving toward *proactive* demand scheduling (Experimental functionality exists)
  - Extract demand within a specified number of computational layers (not including slow-downs due to error correction or finite hardware resources) and *proactively* rather than *reactively* schedule production attempts



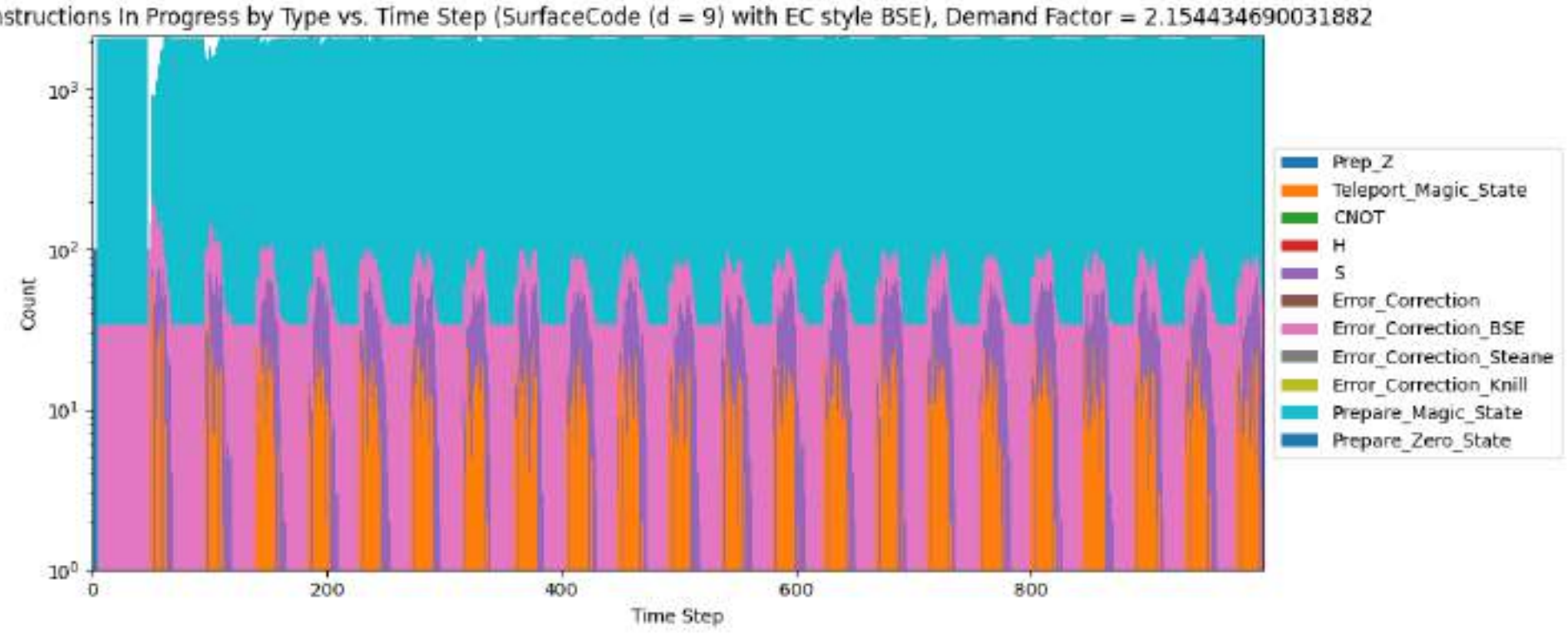
Public
© 2026 Quantinuum. All Rights Reserved.

ReTrace: Scheduling Resource States

Demand-based resource state production scheduling

- In practice, we amplify the instantaneous demand using a “demand factor” knob
- We may also convert to a resource pool maintenance picture using “demand ramping”
- Ex: magic state limited -> T injection limited by increasing the “demand factor”:

  *Helpful for circuits that have parallel layers of many T gates



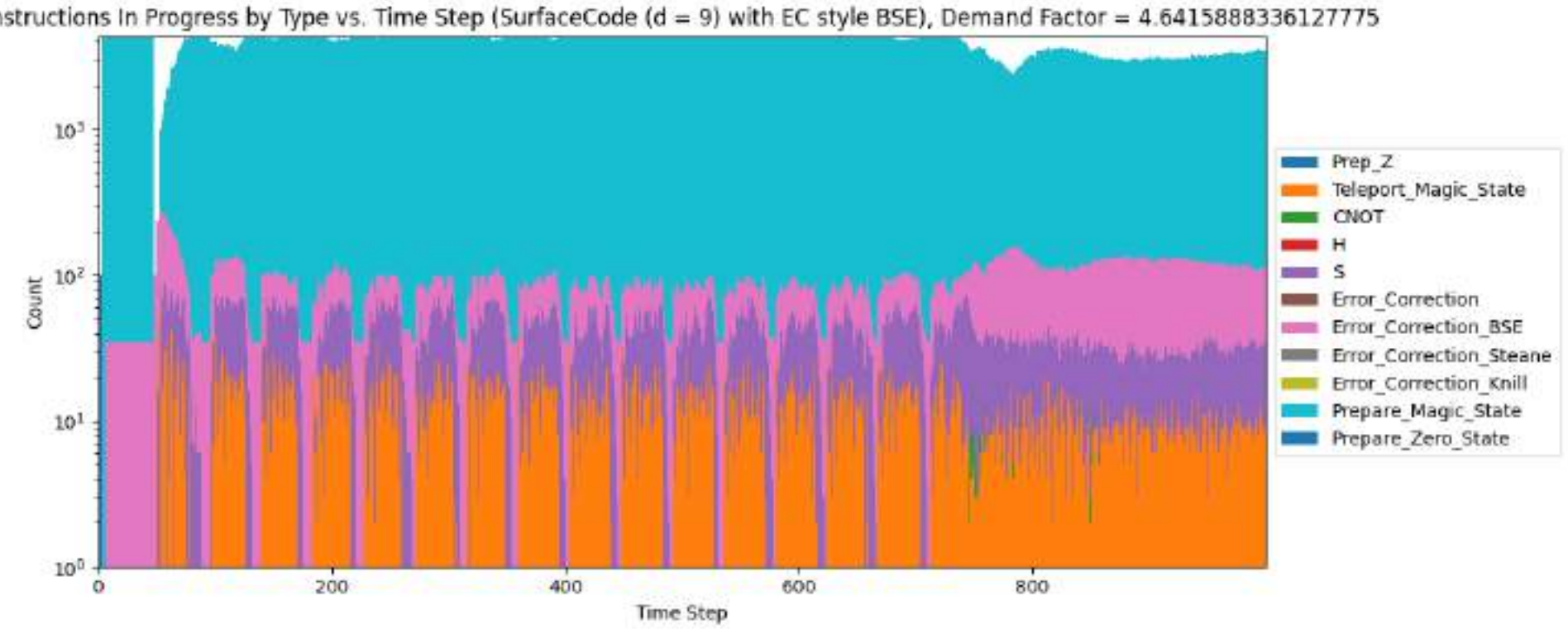
© 2026 Quantinuum. All Rights Reserved.

ReTrace: Scheduling Resource States

Demand-based resource state production scheduling

- In practice, we amplify the instantaneous demand using a “demand factor” knob
- We may also convert to a resource pool maintenance picture using “demand ramping”
- Ex: magic state limited -> T injection limited by increasing the “demand factor”:

  *Helpful for circuits that have parallel layers of many T gates



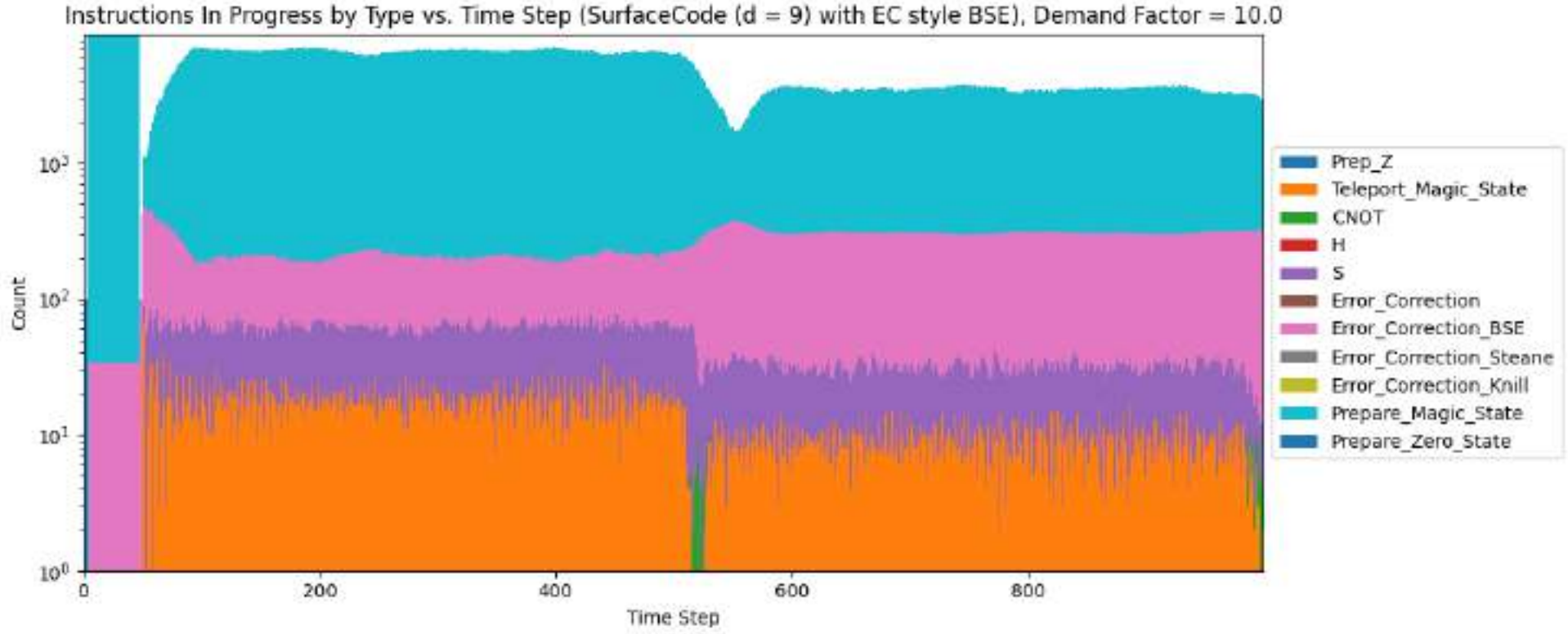
© 2026 Quantinuum. All Rights Reserved.

ReTrace: Scheduling Resource States

Demand-based resource state production scheduling

- In practice, we amplify the instantaneous demand using a “demand factor” knob
- We may also convert to a resource pool maintenance picture using “demand ramping”
- Ex: magic state limited -> T injection limited by increasing the “demand factor”:

*Helpful for circuits that have parallel layers of many T gates



© 2026 Quantinuum. All Rights Reserved.

ReTrace: Resource Estimation via the Logical Execution Trace

Takeaways

We have created a flexible framework for estimating the resource requirements of quantum circuits (time-to-solution, logical error rates) and performing trade-space analyses:

© 2026 Quantinuum. All Rights Reserved.

ReTrace: Resource Estimation via the Logical Execution Trace

Takeaways

We have created a flexible framework for estimating the resource requirements of quantum circuits (time-to-solution, logical error rates) and performing trade-space analyses:

- Encoding rate/addressability trade-off: *where do we benefit from using high-rate codes?*
- Algorithm/QEC co-design: *which gates are our bottleneck and can we avoid them?*
- Distribution of idling times; variability in logical error rate: *where are qubits just waiting?*
- Resource state supply and demand; *the cost of contention and non-determinism*
- Impact of error correction style: *where do we benefit from using resource states for EC?*
- Resource efficiency: *are we maximizing our (spatial) resource utilization?*

Using this framework, we can concretely diagnose resource bottlenecks in QEC architecture design.

© 2026 Quantinuum. All Rights Reserved.

ReTrace: Resource Estimation via the Logical Execution Trace

Takeaways

We have created a flexible framework for estimating the resource requirements of quantum circuits (time-to-solution, logical error rates) and performing trade-space analyses:

- Encoding rate/addressability trade-off: *where do we benefit from using high-rate codes?*
- Algorithm/QEC co-design: *which gates are our bottleneck and can we avoid them?*
- Distribution of idling times; variability in logical error rate: *where are qubits just waiting?*
- Resource state supply and demand; *the cost of contention and non-determinism*
- Impact of error correction style: *where do we benefit from using resource states for EC?*
- Resource efficiency: *are we maximizing our (spatial) resource utilization?*

Using this framework, we can concretely diagnose resource bottlenecks in QEC architecture design.

© 2026 Quantinuum. All Rights Reserved.

Conclusion

- We have developed a tool, **ReTrace**, that helps us analyze this trade space for these arch.
- We have done this in a way that is suitable for trapped ions
- Thank you!

Public
© 2026 Quantinuum. All Rights Reserved.

QUOPS, or:

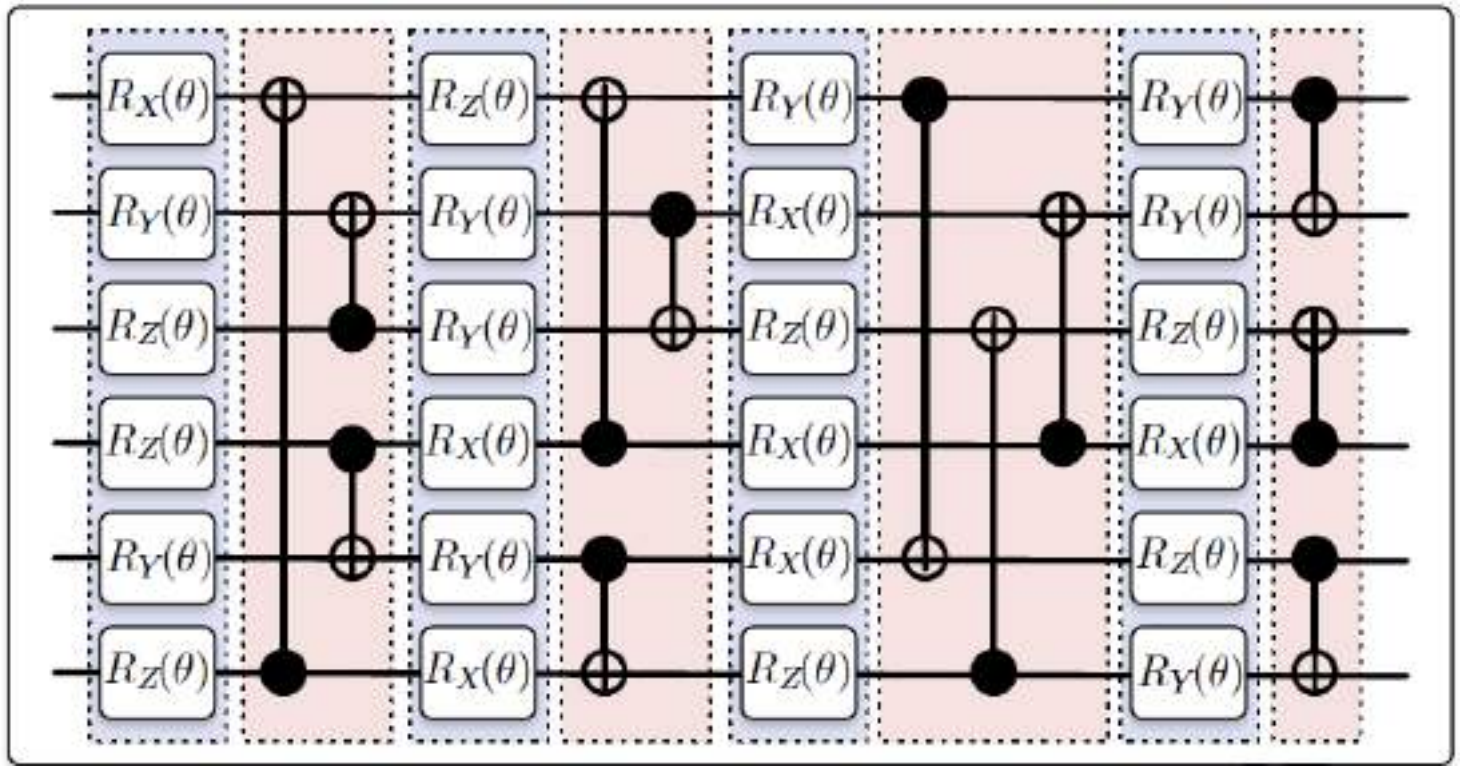
Using ReTrace to determine architecture capability frontiers

Public

© 2026 Quantinuum. All Rights Reserved.

QUOPS Circuit Structure

Definition



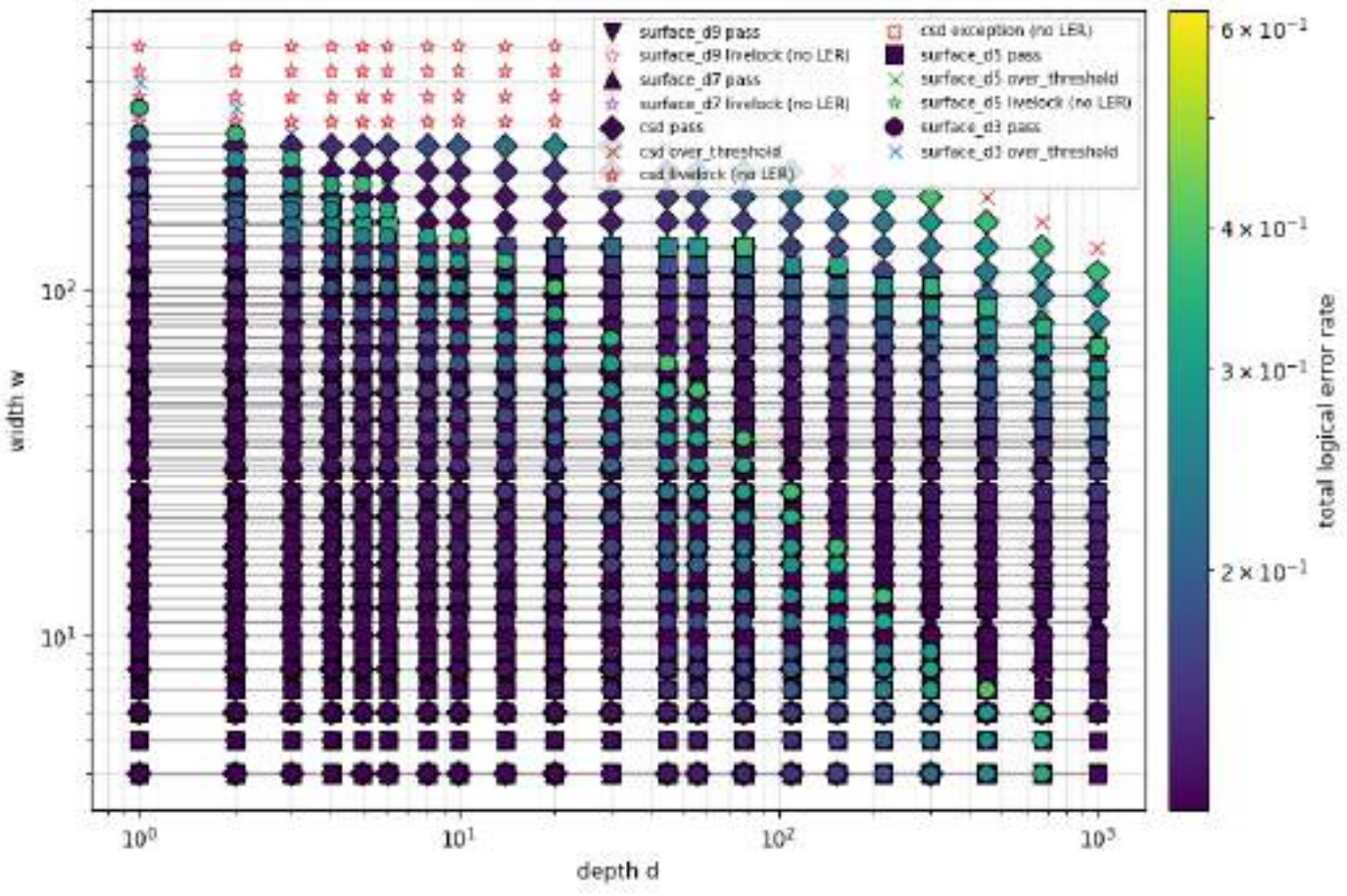
- Alternating layers of random single-qubit rotations and CNOT gates with arbitrary pairings
- Parametrized by “width” (number of logical qubits) and “size” (2*d*w, with ‘d’ the number of Rp/CNOT subunits)
- The QUOPS “capability region”: all (w, s) pairs for which circuit execution results in success > $1/\sqrt{e}$

© 2026 Quantinuum. All Rights Reserved.

QUOPS Architecture Study

Using ReTrace

- Logical Architectures Tried:
  - d=3, 5, 7, 9 surface code w/ BSE
  - [[20, 2, 6]] CSD code w/ BSE
- Physical qubits: Nq = {1000, 5000, 10000}
  - Nq=5000 shown
- Swept a tractable range of (w, d)
- Synthesis error per rotation set to $\sqrt{\frac{(1-1/\sqrt{e})}{(3\mathrm{dw})}}$
  - Assume quadratic synthesis error accumulation due to random circuit structure
- **How do we extrapolate to the whole capability region?**



© 2026 Quantinuum. All Rights Reserved.

QUOPS Architecture Study

Extrapolating to the frontier by fitting an effective model of the scheduler error

- The actual logical error rate calculated by the scheduler is currently:

$$\epsilon_{\text{sched}} = 1 - \left(1 - \epsilon_{\text{mag}}\right)^{n_T} \prod_{i}^{n_{\text{EC}}} \left(1 - \epsilon_{\text{EC}}(m_i)\right)$$

- We approximate this by the following formula (separating EC-due-to-idling from EC-as-a-follow-up):

$$\tilde{\epsilon}_{\text{sched}}(w,d) = 1 - (1 - \epsilon_{\text{eff,mag}})^{n_T} (1 - \epsilon_{\text{eff,EC}})^{\tilde{\tau}_{\text{idle}}(w,d)}$$

$$\tilde{\tau}_{\text{idle}} = \max\left(0, \lceil w/k \rceil \tilde{\tau}_{\text{tot}} - \tau_{\text{injec}} n_T / k\right)$$

$$n_T = w \times d \times K(\log(wd) + \gamma)$$

- This in turn relies on a model for the total number of time steps,

$$\tilde{\tau}_{tot}(w,d) = d \times K(\log(wd) + \gamma) \times \max\left[\tau_{\text{injec}}, w\tau_{\text{prep}}/n_{\text{prep}}(w)\right]$$

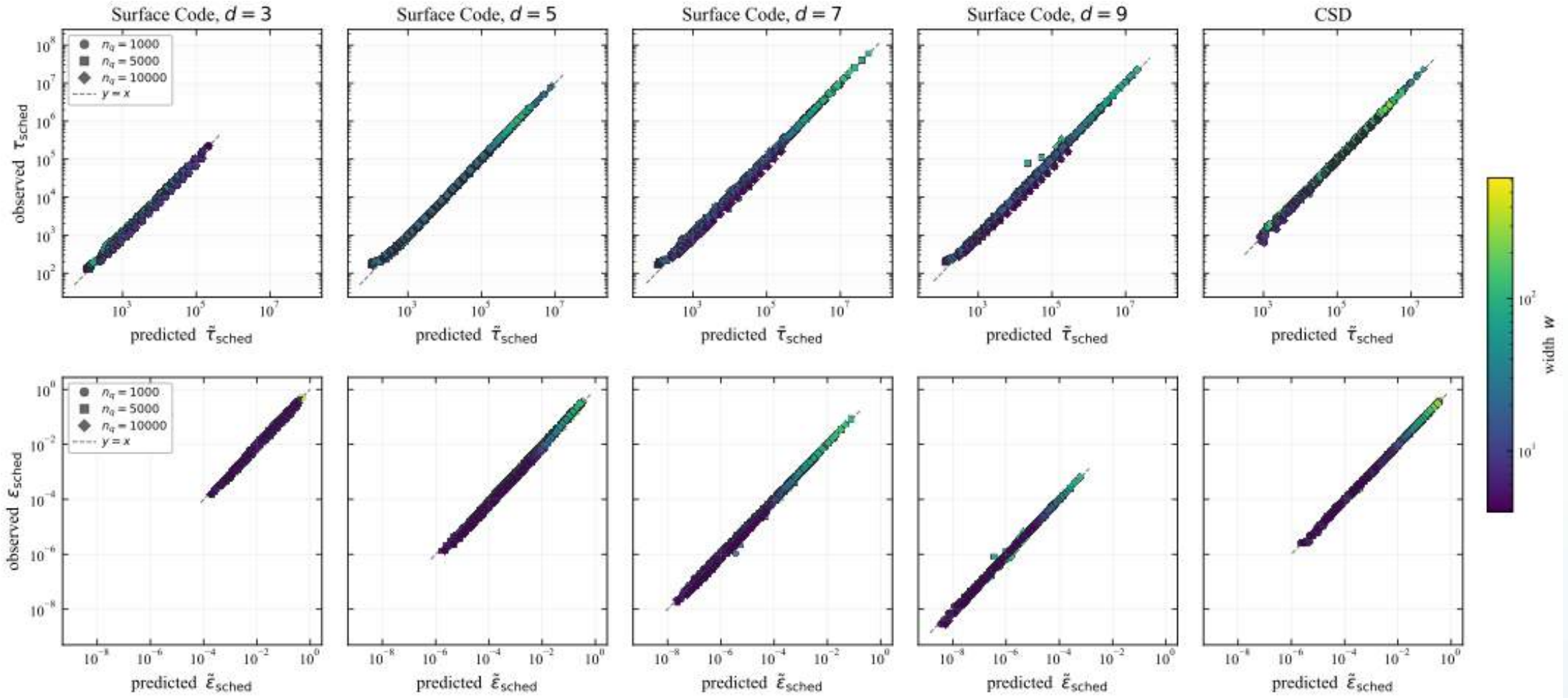
$$n_{\text{prep}}(w) = \max\left(1, \frac{n_q - \tilde{N}_{cb}\left\lceil \frac{w}{k} \right\rceil}{N_{\text{prep}}}\right)$$

- Resource state supply and demand; *the cost of contention and non-determinism*

Public
© 2026 Quantinuum. All Rights Reserved.

QUOPS Architecture Study

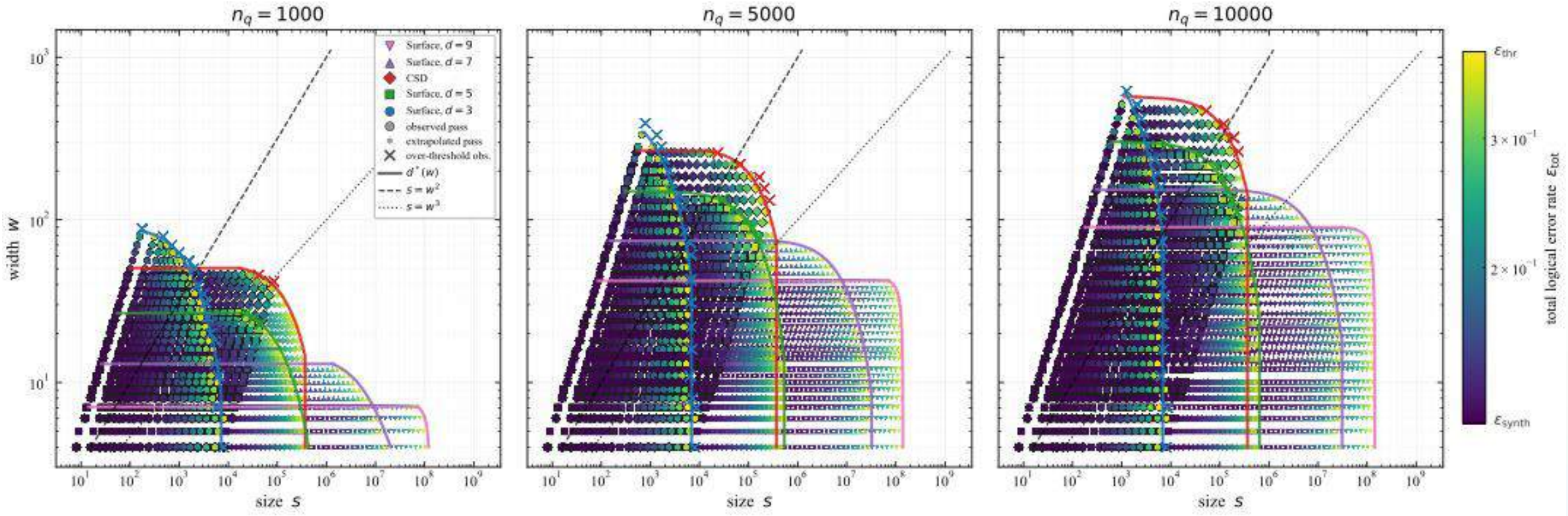
The model fits extremely well to the data



© 2026 Quantinuum. All Rights Reserved.

QUOPS Architecture Study

Observed data + extrapolated data + frontiers + total error rate heatmap

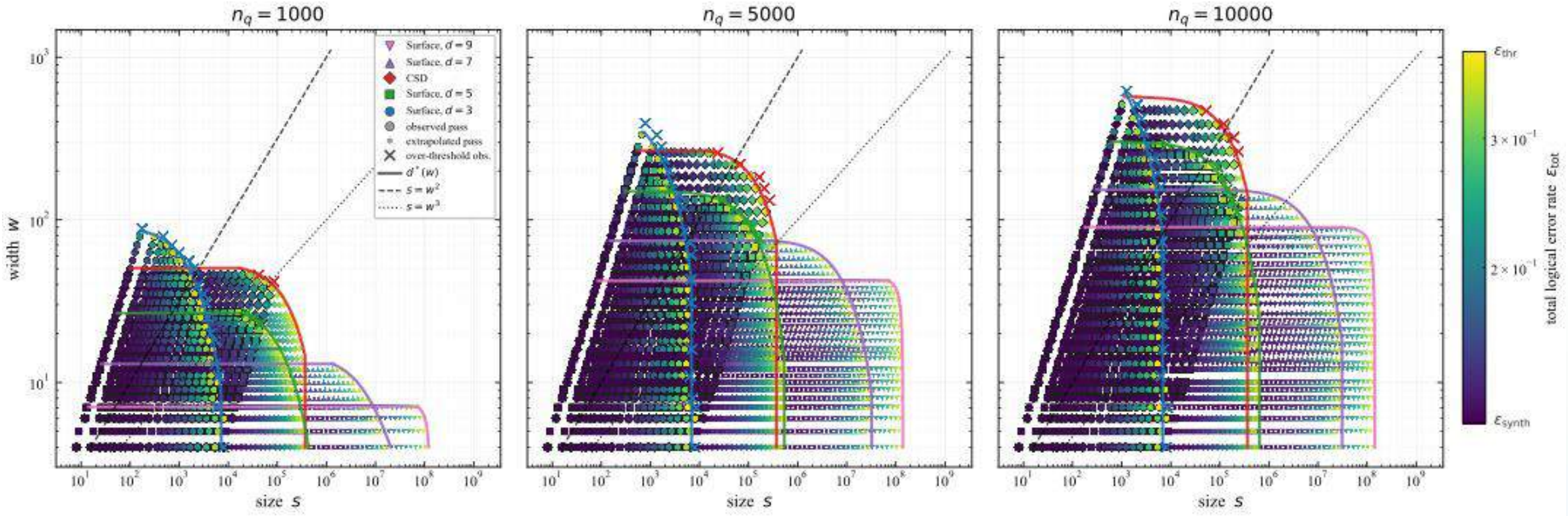


- The $d^*(w)$ lines curves show the maximum circuit size for every width that meets the total error criterion
- The horizontal lines $w_{max} = k(n_q - N_{prep})/\tilde{N}_{cb}$ are the maximum width that allows for a single factory
- Above w_{max}, we find that some low-depth circuits pass, but this is generally a “livelock” zone

Public
© 2026 Quantinuum. All Rights Reserved.

QUOPS Architecture Study

Observed data + extrapolated data + frontiers + total error rate heatmap

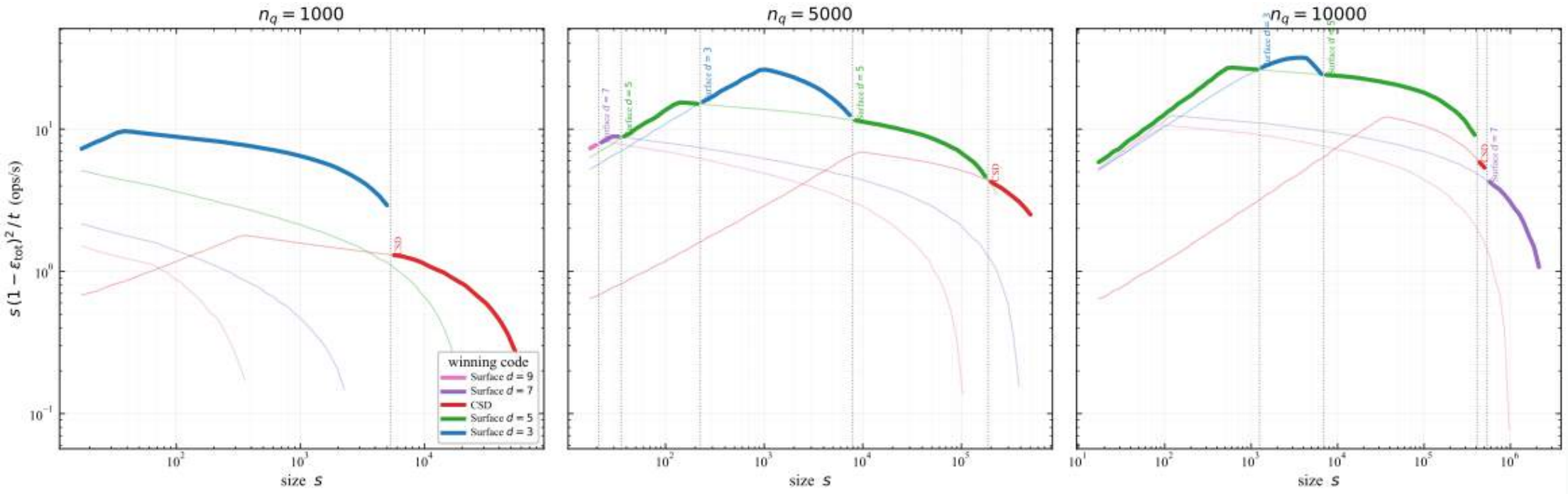


- The $d^*(w)$ lines curves show the maximum circuit size for every width that meets the total error criterion
- The horizontal lines $w_{max} = k(n_q - N_{prep})/\tilde{N}_{cb}$ are the maximum width that allows for a single factory
- Above w_{max}, we find that some low-depth circuits pass, but this is generally a “livelock” zone
- *Result: CSD allows us to access circuits almost as wide as the d=3 surface code but at much larger sizes*

Public
© 2026 Quantinuum. All Rights Reserved.

QUOPS Architecture Study

Throughput plots



- At every circuit size s, we found the throughput-maximizing point *within the utility cone*
- The Nq=5000 device has a maximum throughput of ~30 QUOP/s at a size of ~1e3 QUOPs
  - Recall that QUOPs are single-qubit rotations and CNOTs, not e.g. T gates
  - Goes even higher (~50 QUOP/s) outside of the cone
- Resource efficiency: *are we maximizing our (spatial) resource utilization?*

© 2026 Quantinuum. All Rights Reserved.

Conclusion

- The FTQC trade space is large, and fully worked out FTQC architectures are rare
- We have developed a tool, **ReTrace**, that helps us analyze this trade space for these arch.
- We have done this in a way that is suitable for trapped ions
- Thank you!

Public
© 2026 Quantinuum. All Rights Reserved.